Tutorato 1 - Esercizi Extra

Matteo Mazzaretto

20 Ottobre 2026



Extra 1: cosa stampa?

UNIVERSITÀ DI PADOVA

```cpp
#include <iostream>
using namespace std;

class A {
  int y;
  int x;
public:
  A(int v) : x(v), y(x + 2) {}
  void print() { cout << "x=" << x << ", y=" << y << endl; }
};

int main() {
  A a(5);
  a.print();
  return 0;
}
```

Quale comportamento ci si aspetta e cosa potrebbe stampare `a.print()`?

Extra 1: soluzione

```
Output: x=5, y=<valore_spazzatura_o_undefined>
```

Spiegazione: In C++, i membri di una classe vengono inizializzati **nell'ordine in cui sono dichiarati nella classe**, non nell'ordine della lista di inizializzazione!

- Nella dichiarazione, `y` viene prima di `x`.
- Quindi viene inizializzato prima `y = x + 2` quando `x` non è ancora stato inizializzato (valore indefinito/spazzatura).
- Subito dopo viene inizializzato `x = 5`.

Extra 2

Completa le parti mancanti (`/* 1 */`, `/* 2 */`, `/* 3 */`) dell'operatore di assegnazione della seguente classe `Buffer`:

```
class Buffer {
  int* data;
  int size;
public:
  Buffer& operator=(const Buffer& b) {
    if ( /* 1 */ ) {
      /* 2 */;
      size = b.size;
      data = new int[size];
      for(int i = 0; i < size; ++i) data[i] = b.data[i];
    }
    return /* 3 */;
  }
};
```

Extra 2: soluzione

```
/* 1 */ this != &b
/* 2 */ delete[] data
/* 3 */ *this
```

Spiegazione:

- `/* 1 */`: controllo dell'auto-assegnazione (`this != &b`) per evitare di cancellare la memoria dell'oggetto stesso.
- `/* 2 */`: libera il vecchio array allocato dinamicamente con `delete[] data`.
- `/* 3 */`: ritorna l'oggetto corrente per riferimento (`*this`) per consentire la concatenazione (`b1 = b2 = b3`).

Extra 3: cosa stampa?

UNIVERSITÀ DI PADOVA

```
#include <iostream>
using namespace std;

class Item {
  char id;
public:
  Item(char i) : id(i) { cout << "+" << id << " "; }
  ~Item() { cout << "-" << id << " "; }
};

int main() {
  Item a('A');
  {
    Item b('B');
    static Item c('C');
  }
  Item d('D');
  return 0;
}
```

cosa stampa a schermo l'esecuzione del programma?

Extra 3: soluzione

```
Output: +A +B +C -B +D -D -A -C
```

Spiegazione:

1. `Item a('A');` -> `+A`
2. Entra nel blocco: `Item b('B');` -> `+B`, `static Item c('C');` -> `+C`.
3. Esce dal blocco: `b` viene distrutta -> `-B`. L'oggetto `c` è `static`, quindi vive fino alla fine del programma!
4. `Item d('D');` -> `+D`.
5. Fine del `main`: vengono distrutti in ordine inverso di stack gli oggetti locali `d` (`-D`) e `a` (`-A`).
6. Infine viene distrutto l'oggetto `static c` (`-C`).

Extra 4: cosa stampa?

Il costruttore stampa `C` seguito dall'id, il distruttore stampa `D` seguito dall'id.

```
class D{
  private: int id;
  public:
    D(int i): id(i){ cout << "C" << id << " "; }
    ~D(){ cout << "D" << id << " "; }
};

int main(){
  D a(1);
  {
    D b(2);
    D c(3);
  }
  D d(4);
  cout << "fine ";
}
```

Extra 4: soluzione

Output:

```
C1 C2 C3 D3 D2 C4 fine D4
D1
```

- Gli oggetti locali vengono distrutti all'uscita dal loro blocco, in ordine **inverso** rispetto alla costruzione
- `c` e `b` muoiono alla chiusura del blocco interno (`D3 D2`)
- `d` e `a` muoiono alla fine del `main`, dopo "fine" (`D4 D1`)

Extra 5: cosa stampa?

UNIVERSITÀ DI PADOVA

```
class Acc{
  private: int tot;
  public:
    Acc(): tot(0) {}
    Acc& add(int x){ tot += x; return *this; }
    Acc addCopy(int x){ tot += x; return *this; }
    int get() const { return tot; }
};

int main(){
  Acc a;
  a.add(1).add(2).add(3);
  cout << a.get() << endl;

  Acc b;
  b.addCopy(1).addCopy(2).addCopy(3);
  cout << b.get() << endl;
}
```

Extra 5: soluzione

Output:

```
6
1
```

- `add` ritorna `Acc&`: ogni chiamata della catena agisce sempre sullo stesso oggetto `a`
- `addCopy` ritorna per valore: solo la prima chiamata modifica `b`, le successive modificano una copia temporanea

Extra 6: completa

Completa il file .cpp a partire dall'header.

```
// rect.h
class Rect{
  private:
    double w, h;
  public:
    Rect(double w_in = 1, double
h_in = 1);
    double area() const;
    Rect& scale(double k);
};
```

```
// rect.cpp
______::Rect(double w_in, double h_in):
______ {}

double Rect::area() ______ {
  return w * h;
}

______ Rect::scale(double k){
  w *= k;
  h *= k;
  return ______;
}
```

Extra 6: soluzione

```cpp
// rect.cpp
Rect::Rect(double w_in, double h_in):
w(w_in), h(h_in) {}

double Rect::area() const {
  return w * h;
}

Rect& Rect::scale(double k){
  w *= k;
  h *= k;
  return *this;
}
```

- Il costruttore si definisce con `Rect::Rect` e inizializza i campi nella lista di inizializzazione
- `area` non modifica l'oggetto: `const` va ripetuto anche nella definizione
- `scale` ritorna `Rect&` e `*this`, così si può concatenare
- I valori di default compaiono **solo** nella dichiarazione, non nella definizione

Extra 7: cosa stampa?

```
class Box{
  public:
    int* p;
    Box(int x): p(new int(x)) {}
};

int main(){
  Box a(5);
  Box b = a;
  *b.p = 9;
  cout << *a.p << " " << *b.p << endl;
}
```

Extra 7: soluzione

Output:

```
9 9
```

Il costruttore di copia di default copia il puntatore, non l'intero a cui punta. Come si corregge?

```
class Box{
  public:
    int* p;
    Box(int x): p(new int(x)) {}
    Box(const Box& o): p(new
int(*o.p)) {}
};
```

- `a.p` e `b.p` puntano alla stessa memoria: è una copia **superficiale**
- Nel costruttore di copia ridefinito si alloca un nuovo intero con lo stesso valore (copia **profonda**)
- Con un distruttore che fa `delete p`, la stessa memoria verrebbe liberata due volte

Extra 8: cosa stampa?

```
class M{
  public:
    M(int x){ cout << "int "; }
    M(double x){ cout << "double "; }
    M(int x, int y){ cout << "two "; }
};

int main(){
  M a(3);
  M b(3.5);
  M c(1, 2);
  M d = 4;
}
```

Cosa cambia se il primo costruttore diventa `explicit M(int x)`?

Extra 8: soluzione

Output:

```
int double two int
```

Con `explicit M(int x)`:

```
int double two double
```

- `M a(3)`: corrispondenza esatta con `M(int)`
- `M b(3.5)`: corrispondenza esatta con `M(double)`
- `M c(1, 2)`: `M(int, int)`
- `M d = 4;` usa una conversione implicita: `M(int)`
- Con `explicit`, la conversione implicita non è più possibile con `M(int)`; resta `M(double)` (da `int` a `double`)

Extra 9: completa e stampa

Completa gli spazi e dì cosa stampa il programma.

```
class Money{
  private: int euro;
  public:
    Money(int e = 0): euro(e) {}

    ______ operator+=(const Money& m){
      euro += m.euro;
      return ______;
    }

    int get() const { return euro; }
};

int main(){
  Money a(5), b(3);
  (a += b) += b;
  cout << a.get() << endl;
}
```

Extra 9: soluzione

Completamento:

- tipo di ritorno: `Money&`
- valore ritornato: `*this`

Output:

```
11
```

- `a += b` modifica `a` (8) e ritorna `a` stesso, per riferimento
- il secondo `+= b` agisce ancora su `a`: 8 + 3 = 11
- Se il ritorno fosse per valore, il secondo `+=` agirebbe su una copia temporanea e `a` varrebbe 8

Extra 10: cosa stampa?

```
namespace A{
  int x = 1;
  void f(){ cout << "A::f " << x << endl; }
}
namespace B{
  int x = 2;
  void f(){ cout << "B::f " << x << endl; }
}

int main(){
  A::f();
  B::f();
  using namespace B;
  cout << x << " " << A::x << endl;
}
```

Extra 10: soluzione

Output:

```
A::f 1
B::f 2
2 1
```

- `A::f()` e `B::f()` sono due funzioni diverse, scelte con l'operatore di scoping
- Dopo `using namespace B;`, `x` indica `B::x` (2)
- `A::x` va comunque qualificato, perché `A` non è stato importato (1)